Hypernova 1/6-Scale Prototype: Software & Integration Plan

The software job is to make an ESP32 turn RC/Bluetooth input into a steering-servo angle and a drive-motor throttle, with safe failsafes, by Dec 4. That is mostly bench work on 5V/3.3V parts, so you can start before the battery pack exists.

1. Snapshot: what ECE has, and what you own

The ECE plan fixes a 10S Li-ion pack (37V nominal, 42V full) feeding a power bus, a buck converter for logic, and a solar charge path. Cells, spot-welding and the BMS are the electrical subteam's job; the ESP32, steering, throttle and failsafe logic are yours.

Part (from the list)	Role in the car	Your software touchpoint
AITRIP ESP32 DevKit (3-pack)	Main controller	All firmware; you have 2 spares for bench work
ANNIMOS 150KG servo	Steering	50 Hz PWM, about 500 to 2500 µs pulse
40A RC ESC (3A UBEC)	Drive motor controller	Same servo-style PWM: 1000 to 2000 µs throttle
LA8308 brushless motor	Drive motor (candidate)	None directly; the ESC runs it
A2212 1000KV motor + 30A ESC	Small drive motor (candidate)	Same ESC protocol; use as your bench-test rig
BTS7960 H-bridge (4-pack)	Drives a brushed DC motor only	PWM + direction pins (RPWM / LPWM), only if a brushed motor is used
DC-DC 600W buck	Steps 37V down to 12V or 5V	None; set output with a multimeter first
400W boost converter	Possible solar-to-charge-voltage stage	None, unless ECE adds monitoring
HC-06 Bluetooth	Serial Bluetooth for a microcontroller without Bluetooth	Not needed: the ESP32 has Bluetooth built in
42V 10A charger, XT60s, 12 AWG wire, WAGO 221s	Charging and wiring	None

Red flags to raise with the ECE lead this week

1. **The ESC may not survive the pack.** Listings for the 40A ESC (3A UBEC) show 2 to 6S input, which tops out near 25V. A 10S pack charges to 42V. The ECE plan itself says the ESC must be rated for 10S. Check the spec sheet linked in Discord before anything is powered.
2. **The A2212 1000KV motor is a 2 to 4S drone motor.** On 37V it would spin far past its design speed. Treat it as a low-voltage bench rig only, run from a bench supply or a small 3S pack.
3. **The servo is a 10 to 12.6V part, not 5 to 6V.** The ECE plan routes the servo, receiver and ESP32 all off one 5 to 6V rail. The servo needs its own 12V rail from the 600W buck (check its exact range on the spec sheet), and the ESP32 needs a separate 5V rail. A large servo can pull several amps when it stalls.
4. **The LA8308 is a large agricultural-drone motor** (KV 90 to 205 by variant, per the Amazon listing). Confirm which KV you have and whether it makes sense for a 1/6-scale car.
5. **Missing from the list:** the 10S BMS itself, the cells, a solar charge controller, a fuse, an e-stop switch, and an RC transmitter and receiver. Ask whether these are ordered.
6. **Scale mismatch:** you said 1/16, the ECE doc says 1/6. Confirm which, because it changes motor and servo sizing.

I could not open the Amazon pages for the ESC or servo, so items 1 and 3 come from search listings. Treat them as "verify", not fact.

3. Breadboard, solder, or neither

Use a breadboard while you develop, never on the car, and never for anything that carries motor or battery current. Breadboards are only good for about 1A, and the vibration on a moving car pops jumper wires loose.

Connection	Method	Why
ESP32 to servo and ESC signal wires (bench phase)	Breadboard + jumpers	Fast to change while you tune code
ESP32 to servo and ESC signal wires (on the car)	Soldered perfboard, or an ESP32 screw-terminal breakout board	Survives vibration; the breakout needs no soldering
Battery, ESC, motor, buck converter power	XT60 connectors + WAGO 221 lever nuts	High current; WAGOs need no solder
XT60 plugs onto 12 AWG wire	Solder (cup terminals)	Crimp-free, and this is the one joint everyone has to learn
Pin headers onto perfboard	Solder	Only if you build a permanent control board
Battery voltage sensing (optional)	Resistor divider, soldered on perfboard	Lets the ESP32 read pack voltage

Connection	Method	Why
Cell-to-cell nickel strips	Spot weld (ECE does this)	Soldering cells can damage them

A few habits that prevent the usual first-timer failures:

- **Common ground.** The ESP32, ESC, servo supply and receiver must all share a ground, or the signal wires do nothing.
- **Never power the ESP32 from USB and the car rail at once** unless the board has a diode on its 5V pin. Unplug USB when the car rail is live.
- **Set the buck converter with a multimeter before connecting anything.** Adjustable modules ship with arbitrary output voltages.
- **Bench first on a low-voltage supply.** Test the A2212 + 30A ESC on 3S or a bench supply before the 10S pack ever exists.
- **Wheels off the ground** for every first power-on.

A 30-pin ESP32 DevKit is wide and covers most of a standard breadboard. Buy a full-size 830-point board, or two half-size boards, so the pins have free rows beside them. Sizing the divider if you add voltage sensing: 150k from pack plus to the ADC pin and 10k from the pin to ground gives about 2.6V at 42V, safely under the ESP32's 3.3V limit.

4. Week-by-week software plan

Your work runs one step ahead of the ECE schedule, so a tested control stack is waiting when their power path is wired on Nov 6. Dates follow the Friday 3 to 5 PM meetings in the ECE plan.

Friday	You build	ECE is doing	Done when
Oct 2	Install Arduino IDE or PlatformIO, ESP32 board support; blink an LED; send the red flags to the ECE lead	Sizing the pack	Serial monitor prints from your ESP32
Oct 9	Drive a hobby servo with 50 Hz PWM from the ESP32; sweep with enforced min/max limits	Solar and power architecture	Servo sweeps 1000 to 2000 µs smoothly with no jitter
Oct 16	Arm an ESC and spin the A2212 on a bench supply: throttle ramp, neutral, brake	Cell testing, spot-welding	Motor follows a commanded throttle and ramps without jumps

Friday	You build	ECE is doing	Done when
Oct 23	Add input: phone over Bluetooth, or RC receiver PWM (decide with ECE)	Pack enclosed, first charge	One input source steers and throttles on the bench
Oct 30	Failsafes: signal-loss timeout to neutral, arming sequence, hardware e-stop input, watchdog	Wiring charging path	Pulling the signal or hitting e-stop stops the motor within 0.5 s
Nov 6	Integrate on the chassis with wheels off the ground; calibrate steering end stops	Wiring power bus, ESC, buck converter	Car responds on real power, no binding at full lock
Nov 13	Move from breadboard to perfboard or breakout board; strain-relieve every wire	Wiring control path	Harness survives a shake test with no dropouts
Nov 20	Join the ECE validation checklist; log loop timing and brownouts	Voltage, BMS cutoff, load tests	ESC, servo, receiver and ESP32 all power on cleanly under load
Nov 27	No meeting; write up code and wiring notes	Thanksgiving	README drafted
Dec 4	Floor tests: steering response, at least 3 mph, tune throttle curve; tag the code release	Fuse and e-stop check, freeze docs	Car holds 3 mph and stops on e-stop
Dec 11	Buffer or demo; handoff notes for the full-size car	Optional buffer	Code tagged, notes shared with next semester's team

The one real schedule risk is the ESC voltage rating. If the 40A ESC cannot take 10S, ECE needs a replacement ESC before Nov 6, so raise it now.

5. What to learn, in order

You need eight skills, and the first three get you to a spinning motor. Each row says what to learn and where to start. The sites are well-known beginner sources; I named the tutorial topics from memory, so search the site for the title if a link has moved.

#	Learn	Why you need it	Start here	Time
1	Electronics basics: voltage, current, ground, multimeter use, voltage dividers	Prevents burned parts; you will check every rail by hand	SparkFun Learn: "Voltage, Current, Resistance, and Ohm's Law", "How to Use a Multimeter", "Voltage Dividers"	3 to 4 hrs
2	ESP32 + Arduino or PlatformIO toolchain, pinouts, which pins are safe	Everything else runs on it	Random Nerd Tutorials: "Getting Started with ESP32" and the pinout reference; PlatformIO docs	4 hrs
3	RC PWM signals and the ESP32 LEDC peripheral	How you command both the servo and the ESC (50 Hz, 1000 to 2000 µs)	Arduino-ESP32 docs: LEDC; ESP32Servo library; Random Nerd Tutorials "ESP32 Servo Motor"	4 hrs
4	ESC arming, calibration and safety	An ESC ignores you until armed, and can spin a motor hard if you skip calibration	DroneBot Workshop on YouTube (brushless motors + ESC); Oscar Liang ESC articles	3 hrs
5	Wireless input: BLE (iPhones cannot use Bluetooth Classic) or reading an RC receiver	The driver's link to the car	Random Nerd Tutorials "ESP32 Bluetooth Low Energy"; nRF Connect app for testing; for receivers, the Arduino "pulseIn"	6 hrs

#	Learn	Why you need it	Start here	Time
			reference and ESP32 interrupts	
6	Non-blocking code, timers, and FreeRTOS tasks	Needed so failsafe checks keep running while input is read	Digi-Key "Introduction to RTOS" (Shawn Hymel, YouTube); Arduino "Blink Without Delay"	6 hrs
7	Soldering (through-hole, XT60)	Final harness and perfboard	Adafruit Guide to Excellent Soldering; ask the ECE team for a supervised session	2 hrs + practice
8	Git for team firmware	Several people will touch the same code	GitHub's "Hello World" guide; one repo, one branch per feature	2 hrs

What to skip for now: PID control, CAN bus, and solar MPPT algorithms. They matter for the full-size car next semester, not this prototype. Note them in the handoff doc instead.

Safety learning is not optional. Before the first powered test, read how Li-ion packs fail (short circuits, overcharge, damaged cells) and ask ECE to walk you through the e-stop and fuse. The 37V bus can deliver enough current to start a fire.

6. Starter firmware and bench test

This skeleton is your Oct 9 to Oct 30 target in one file: steering and throttle from serial commands, a throttle ramp, a power-on arming delay, a signal-loss timeout, and an e-stop input. I have not run it on hardware, so treat it as a starting point and check the pin choices against your board's pinout.

```
#include <ESP32Servo.h>

const int PIN_STEER = 18, PIN_ESC = 19, PIN_ESTOP = 27;
const int STEER_MIN_US = 1000, STEER_MAX_US = 2000;   // narrow until you find the
end stops
const int ESC_STOP_US = 1000, ESC_MAX_US = 1300;      // 1500 = stop if the ESC is
bidirectional; 1300 caps first tests
const int RAMP_US_PER_TICK = 5;
const unsigned long TIMEOUT_MS = 500, TICK_MS = 20;

Servo steer, esc;
int steerUs = 1500, targetUs = ESC_STOP_US, currentUs = ESC_STOP_US;
unsigned long lastCmdMs = 0, lastTickMs = 0;
```

```
void setup() {
  Serial.begin(115200);
  pinMode(PIN_ESTOP, INPUT_PULLUP);   // normally-closed switch to GND: open or
broken wire = stop
  steer.setPeriodHertz(50);
  esc.setPeriodHertz(50);
  steer.attach(PIN_STEER, 500, 2500);
  esc.attach(PIN_ESC, 1000, 2000);
  steer.writeMicroseconds(steerUs);
  esc.writeMicroseconds(ESC_STOP_US);
  delay(3000);                        // let the ESC arm at minimum throttle
  lastCmdMs = millis();
}

void loop() {
  // Serial commands: "s1500" sets steering, "t1100" sets throttle
  if (Serial.available()) {
    char c = Serial.read();
    int v = Serial.parseInt();
    if (c == 's') steerUs = constrain(v, STEER_MIN_US, STEER_MAX_US);
    if (c == 't') targetUs = constrain(v, ESC_STOP_US, ESC_MAX_US);
    lastCmdMs = millis();
  }

  bool estop = digitalRead(PIN_ESTOP) == HIGH;
  bool stale = millis() - lastCmdMs > TIMEOUT_MS;
  if (estop || stale) targetUs = ESC_STOP_US;

  if (millis() - lastTickMs >= TICK_MS) {
    lastTickMs = millis();
    if (currentUs < targetUs) currentUs = min(currentUs + RAMP_US_PER_TICK,
targetUs);
    else if (currentUs > targetUs) currentUs = targetUs;  // slowing down is
immediate; only speeding up is ramped
    esc.writeMicroseconds(currentUs);
    steer.writeMicroseconds(steerUs);
  }
}
```

The signal-loss timeout means a serial command must repeat at least every 0.5 s to keep the motor running, which is also how your phone app should behave later. Swap the serial block for BLE or receiver input on Oct 23 without touching the safety logic.

First bench test, in order

1. Power the ESP32 over USB only. Confirm the servo and ESC signal pins output 50 Hz with a cheap logic analyzer or oscilloscope if the lab has one.
2. Power the servo from its own supply at the voltage on its spec sheet, with ground tied to the ESP32 ground. Send `s1500`, then small steps. Stop if it buzzes against an end stop.
3. Connect the A2212 and its 30A ESC to a bench supply or a small 3S pack, propeller off, motor clamped down. Power the ESC after the ESP32 has booted.
4. Wait for the arming beeps, then send `t1050`. Raise the value in small steps.
5. Stop sending commands and confirm the motor cuts within about half a second.

6. Open the e-stop circuit and confirm the same.
7. Only then move to the chassis, wheels off the ground.

7. Questions to bring to the ECE lead

- ☐ Is the scale 1/6 (ECE doc) or 1/16 (your brief)?
- ☐ Can the 40A ESC take a 10S pack (42V)? If not, which ESC replaces it, and when does it arrive?
- ☐ Which drive motor is final: LA8308 (which KV), A2212, or something else? Is it brushless-only, so the BTS7960 H-bridges go unused?
- ☐ What voltage range does the ANNIMOS servo accept, and will the 600W buck supply its own 12V rail?
- ☐ Are the BMS, cells, solar charge controller, fuse, e-stop switch, and RC transmitter and receiver ordered?
- ☐ Decision on control input: RC receiver into the ESP32, phone over BLE, or both?
- ☐ Is the ESP32 fed from its own 5V rail, and who owns that wiring?
- ☐ Where is the Discord spec-sheet link, and who can give me supervised soldering time?